CHAPTER 10

Characters, C-Strings, and More about the `string` Class

TOPICS

10.1 Character Testing
10.2 Character Case Conversion
10.3 C-Strings
10.4 Library Functions for Working with C-Strings
10.5 String/Numeric Conversion Functions
10.6 Focus on Software Engineering: Writing Your Own C-String-Handling Functions
10.7 More about the C++ `string` Class
10.8 Focus on Problem Solving and Program Design: A Case Study

10.1 Character Testing

CONCEPT: The C++ library provides several functions for testing characters. To use these functions you must include the <cctype> header file.

The C++ library provides several functions that allow you to test the value of a character. These functions test a single `char` argument and return either `true` or `false`.* For example, the following program segment uses the `isupper` function to determine whether the character passed as an argument is an uppercase letter. If it is, the function returns `true`. Otherwise, it returns `false`.

```
char letter = 'a';
if (isupper(letter))
   cout << "Letter is uppercase.\n";
else
   cout << "Letter is lowercase.\n";
```

Because the variable `letter`, in this example, contains a lowercase character, `isupper` returns `false`. The `if` statement will cause the message `"Letter is lowercase"` to be displayed.

* These functions actually return an `int` value. The return value is nonzero to indicate `true`, or zero to indicate `false`.

Table 10-1 lists several character-testing functions. Each of these is prototyped in the <cctype> header file, so be sure to include that file when using the functions.

Table 10-1 Character-Testing Functions in <cctype>

Character Function	Description
isalpha	Returns true (a nonzero number) if the argument is a letter of the alphabet. Returns 0 if the argument is not a letter.
isalnum	Returns true (a nonzero number) if the argument is a letter of the alphabet or a digit. Otherwise, it returns 0.
isdigit	Returns true (a nonzero number) if the argument is a digit from 0 through 9. Otherwise, it returns 0.
islower	Returns true (a nonzero number) if the argument is a lowercase letter. Otherwise, it returns 0.
isprint	Returns true (a nonzero number) if the argument is a printable character (including a space). Returns 0 otherwise.
ispunct	Returns true (a nonzero number) if the argument is a printable character other than a digit, letter, or space. Returns 0 otherwise.
isupper	Returns true (a nonzero number) if the argument is an uppercase letter. Otherwise, it returns 0.
isspace	Returns true (a nonzero number) if the argument is a whitespace character. Whitespace characters are any of the following: space ' ' vertical tab '\v' newline '\n' tab '\t' Otherwise, it returns 0.

Program 10-1 uses several of the functions shown in Table 10-1. It asks the user to input a character then displays various messages, depending upon the return value of each function.

Program 10-1

```
1  // This program demonstrates some character-testing functions.
2  #include <iostream>
3  #include <cctype>
4  using namespace std;
5
6  int main()
7  {
8      char input;
9
10     cout << "Enter any character: ";
11     cin.get(input);
12     cout << "The character you entered is: " << input << endl;
```

```
13      if (isalpha(input))
14         cout << "That's an alphabetic character.\n";
15      if (isdigit(input))
16         cout << "That's a numeric digit.\n";
17      if (islower(input))
18         cout << "The letter you entered is lowercase.\n";
19      if (isupper(input))
20         cout << "The letter you entered is uppercase.\n";
21      if (isspace(input))
22         cout << "That's a whitespace character.\n";
23      return 0;
24  }
```

Program Output with Example Input Shown in Bold

```
Enter any character: A Enter
The character you entered is: A
That's an alphabetic character.
The letter you entered is uppercase.
```

Program Output with Different Example Input Shown in Bold

```
Enter any character: 7 Enter
The character you entered is: 7
That's a numeric digit.
```

Program 10-2 shows a more practical application of the character-testing functions. It tests a seven-character customer number to determine whether it is in the proper format.

Program 10-2

```
1   // This program tests a customer number to determine whether
2   // it is in the proper format.
3   #include <iostream>
4   #include <cctype>
5   using namespace std;
6
7   // Function prototype
8   bool testNum(char [], int);
9
10  int main()
11  {
12      const int SIZE = 8;    // Array size
13      char customer[SIZE];   // To hold a customer number
14
15      // Get the customer number.
16      cout << "Enter a customer number in the form ";
17      cout << "LLLNNNN\n";
18      cout << "(LLL = letters and NNNN = numbers): ";
19      cin.getline(customer, SIZE);
20
```

(program continues)

Program 10-2 *(continued)*

```
21      // Determine whether it is valid.
22      if (testNum(customer, SIZE))
23          cout << "That's a valid customer number.\n";
24      else
25      {
26          cout << "That is not the proper format of the ";
27          cout << "customer number.\nHere is an example:\n";
28          cout << "    ABC1234\n";
29      }
30      return 0;
31  }
32
33  //*********************************************************
34  // Definition of function testNum.                        *
35  // This function determines whether the custNum parameter *
36  // holds a valid customer number. The size parameter is   *
37  // the size of the custNum array.                         *
38  //*********************************************************
39
40  bool testNum(char custNum[], int size)
41  {
42      int count; // Loop counter
43
44      // Test the first three characters for alphabetic letters.
45      for (count = 0; count < 3; count++)
46      {
47          if (!isalpha(custNum[count]))
48              return false;
49      }
50
51      // Test the remaining characters for numeric digits.
52      for (count = 3; count < size - 1; count++)
53      {
54          if (!isdigit(custNum[count]))
55              return false;
56      }
57      return true;
58  }
```

Program Output with Example Input Shown in Bold

```
Enter a customer number in the form LLLNNNN
(LLL = letters and NNNN = numbers): RQS4567 Enter
That's a valid customer number.
```

Program Output with Different Example Input Shown in Bold

```
Enter a customer number in the form LLLNNNN
(LLL = letters and NNNN = numbers): AX467T9 Enter
That is not the proper format of the customer number.
Here is an example:
    ABC1234
```

In this program, the customer number is expected to consist of three alphabetic letters followed by four numeric digits. The `testNum` function accepts an array argument and tests the first three characters with the following loop in lines 45 through 49:

```
for (count = 0; count < 3; count++)
{
     if (!isalpha(custNum[count]))
        return false;
}
```

The `isalpha` function returns `true` if its argument is an alphabetic character. The `!` operator is used in the `if` statement to determine whether the tested character is NOT alphabetic. If this is so for any of the first three characters, the function `testNum` returns `false`. Likewise, the next four characters are tested to determine whether they are numeric digits with the following loop in lines 52 through 56:

```
for (count = 3; count < size - 1; count++)
{
     if (!isdigit(custNum[count]))
        return false;
}
```

The `isdigit` function returns `true` if its argument is the character representation of any of the digits 0 through 9. Once again, the `!` operator is used to determine whether the tested character is *not* a digit. If this is so for any of the last four characters, the function `testNum` returns `false`. If the customer number is in the proper format, the function will cycle through both the loops without returning `false`. In that case, the last line in the function is the `return true` statement, which indicates the customer number is valid.

10.2 Character Case Conversion

CONCEPT: The C++ library offers functions for converting a character to uppercase or lowercase.

The C++ library provides two functions, `toupper` and `tolower`, for converting the case of a character. The functions are described in Table 10-2. (These functions are prototyped in the header file `<cctype>`, so be sure to include it.)

Table 10-2 Case-Conversion Functions in `<cctype>`

Function	Description
`toupper`	Returns the uppercase equivalent of its argument.
`tolower`	Returns the lowercase equivalent of its argument.

Each of the functions in Table 10-2 accepts a single character argument. If the argument is a lowercase letter, the `toupper` function returns its uppercase equivalent. For example, the following statement will display the character `A` on the screen:

```
cout << toupper('a');
```

If the argument is already an uppercase letter, toupper returns it unchanged. The following statement causes the character Z to be displayed:

```
cout << toupper('Z');
```

Any nonletter argument passed to toupper is returned as it is. Each of the following statements displays toupper's argument without any change:

```
cout << toupper('*');    // Displays *
cout << toupper ('&');   // Displays &
cout << toupper('%');    // Displays %
```

toupper and tolower don't actually cause the character argument to change; they simply return the uppercase or lowercase equivalent of the argument. For example, in the following program segment, the variable letter is set to the value 'A'. The tolower function returns the character 'a', but letter still contains 'A'.

```
char letter = 'A';
cout << tolower(letter) << endl;
cout << letter << endl;
```

These statements will cause the following to be displayed:

```
a
A
```

Program 10-3 demonstrates the toupper function in an input validation loop.

Program 10-3

```
1  // This program calculates the area of a circle. It asks the user
2  // if he or she wishes to continue. A loop that demonstrates the
3  // toupper function repeats until the user enters 'y', 'Y',
4  // 'n', or 'N'.
5  #include <iostream>
6  #include <cctype>
7  #include <iomanip>
8  using namespace std;
9
10 int main()
11 {
12     const double PI = 3.14159;  // Constant for pi
13     double radius;              // The circle's radius
14     char goAgain;               // To hold Y or N
15
16     cout << "This program calculates the area of a circle.\n";
17     cout << fixed << setprecision(2);
18
19     do
20     {
21         // Get the radius and display the area.
22         cout << "Enter the circle's radius: ";
23         cin >> radius;
24         cout << "The area is " << (PI * radius * radius);
25         cout << endl;
```

```
26
27          // Does the user want to do this again?
28          cout << "Calculate another? (Y or N) ";
29          cin >> goAgain;
30
31          // Validate the input.
32          while (toupper(goAgain) != 'Y' && toupper(goAgain) != 'N')
33          {
34               cout << "Please enter Y or N: ";
35               cin >> goAgain;
36          }
37
38      } while (toupper(goAgain) == 'Y');
39      return 0;
40  }
```

Program Output with Example Input Shown in Bold

```
This program calculates the area of a circle.
Enter the circle's radius: 10 Enter
The area is 314.16
Calculate another? (Y or N) b Enter
Please enter Y or N: y Enter
Enter the circle's radius: 1 Enter
The area is 3.14
Calculate another? (Y or N) n Enter
```

In lines 28 and 29, the user is prompted to enter either Y or N to indicate whether he or she wants to calculate another area. We don't want the program to be so picky that it accepts only uppercase Y or uppercase N. Lowercase y or lowercase n is also acceptable. The input validation loop must be written to reject anything except `'Y'`, `'y'`, `'N'`, or `'n'`. One way to do this would be to test the `goAgain` variable in four relational expressions, as shown here:

```
while (goAgain != 'Y' && goAgain != 'y' &&
       goAgain != 'N' && goAgain != 'N')
```

Although there is nothing wrong with this code, we could use the `toupper` function to get the uppercase equivalent of `goAgain` and make only two comparisons. This is the approach taken in line 32:

```
while (toupper(goAgain) != 'Y' && toupper(goAgain) != 'N')
```

Another approach would have been to use the `tolower` function to get the lowercase equivalent of `goAgain`. Here is an example:

```
while (tolower(goAgain) != 'y' && tolower(goAgain) != 'n')
```

Either approach will yield the same results.



Checkpoint

10.1 Write a short description of each of the following functions:

```
isalpha
isalnum
isdigit
islower
isprint
ispunct
isupper
isspace
toupper
tolower
```

10.2 Write a statement that will convert the contents of the `char` variable `big` to lowercase. The converted value should be assigned to the variable `little`.

10.3 Write an `if` statement that will display the word "digit" if the variable `ch` contains a numeric digit. Otherwise, it should display "Not a digit."

10.4 What is the output of the following statement?

```
cout << toupper(tolower('A'));
```

10.5 Write a loop that asks the user "Do you want to repeat the program or quit? (R/Q)". The loop should repeat until the user has entered an R or a Q (either uppercase or lowercase).

10.3 C-Strings

CONCEPT: In C++, a C-string is a sequence of characters stored in consecutive memory locations, terminated by a null character.

String is a generic term that describes any consecutive sequence of characters. A word, a sentence, a person's name, and the title of a song are all strings. In the C++ language, there are two primary ways that strings are stored in memory: as `string` objects or as C-strings. You have already been introduced to the `string` class, and by now, you have written several programs that use `string` objects. In this section, we will use C-strings, which are an alternative method for storing and working with strings.

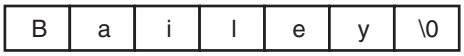
A *C-string* is a string whose characters are stored in consecutive memory locations and are followed by a null character, or null terminator. Recall from Chapter 2 that a null character or null terminator is a byte holding the ASCII code 0. Strings that are stored this way are called C-strings because this is the way strings are handled in the C programming language.

In C++, all string literals are stored in memory as C-strings. Recall that a string literal (or string constant) is the literal representation of a string in a program. In C++, string literals are enclosed in double quotation marks, such as:

```
"Bailey"
```

Figure 10-1 illustrates how the string literal `"Bailey"` is stored in memory, as a C-string.

Figure 10-1 A C-string stored in memory





NOTE: Remember that `\0` ("slash zero") is the escape sequence representing the null terminator. It stands for the ASCII code 0.

The purpose of the null terminator is to mark the end of the C-string. Without it, there would be no way for a program to know the length of a C-string.

More about String Literals

A string literal or string constant is enclosed in a set of double quotation marks (`" "`). For example, here are five string literals:

```
"Have a nice day."
"What is your name?"
"John Smith"
"Please enter your age:"
"Part Number 45Q1789"
```

All of a program's string literals are stored in memory as C-strings, with the null terminator automatically appended. For example, look at Program 10-4.

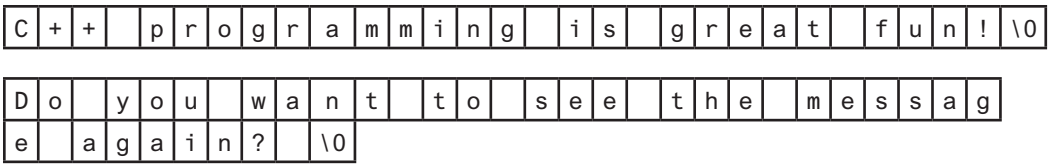
Program 10-4

```
1  // This program contains string literals.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      char again;
8
9      do
10     {
11           cout << "C++ programming is great fun!" << endl;
12           cout << "Do you want to see the message again? ";
13           cin >> again;
14     } while (again == 'Y' || again == 'y');
15     return 0;
16 }
```

This program contains two string literals:

```
"C++ programming is great fun!"
"Do you want to see the message again? "
```

The first string occupies 30 bytes of memory (including the null terminator), and the second string occupies 39 bytes. They appear in memory in the following forms:



It's important to realize that a string literal has its own storage location, just like a variable or an array. When a string literal appears in a statement, it's actually its memory address that C++ uses. Look at the following example:

```
cout << "Do you want to see the message again? ";
```

In this statement, the memory address of the string literal "Do you want to see the message again?" is passed to the `cout` object. `cout` displays the consecutive characters found at this address. It stops displaying the characters when a null terminator is encountered.

C-Strings Stored in Arrays

The C programming language does not provide a `string` class like that which C++ provides. In the C language, all strings are treated as C-strings. When a C programmer wants to store a string in memory, he or she has to create a `char` array that is large enough to hold the string, plus one extra element for the null character.

You might be wondering why this should matter to anyone learning C++. You need to know about C-strings for the following reasons:

- The `string` class has not always existed in the C++ language. Several years ago, C++ stored strings as C-strings. As a professional programmer, you might encounter older C++ code (known as *legacy code*) that uses C-strings.
- Some of the C++ library functions work only with C-strings.
- In the workplace, it is not unusual for C++ programmers to work with specialized libraries that are written in C. Any strings with which C libraries work will be C-strings.

As previously mentioned, if you want to store a C-string in memory, you have to define a `char` array that is large enough to hold the string, plus one extra element for the null character. Here is an example:

```
const int SIZE = 21;
char name[SIZE];
```

This code defines a `char` array that has 21 elements, so it is big enough to hold a C-string that is no more that 20 characters long.

You can initialize a `char` array with a string literal, as shown here:

```
const int SIZE = 21;
char name[SIZE] = "Jasmine";
```

After this code executes, the `name` array will be created with 21 elements. The first eight elements will be initialized with the characters `'J'`, `'a'`, `'s'`, `'m'`, `'i'`, `'n'`, `'e'`, and `'\0'`. The null character is automatically added as the last character. You can implicitly size a `char` array by initializing it with a string literal, as shown here:

```
char name[] = "Jasmine";
```

After this code executes, the name array will be created with eight elements, initialized with the characters 'J','a', 's', 'm', 'i', 'n', 'e', and '\0'.

C-string input can be performed by the cin object. For example, the following code allows the user to enter a string (with no whitespace characters) into the name array:

```
const int SIZE = 21;
char name[SIZE];
cin >> name;
```

Recall from Chapter 7 that an array name with no brackets and no subscript is converted into the beginning address of the array. In the previous statement, name indicates the address in memory where the string is to be stored. Of course, cin has no way of knowing that name has 21 elements. If the user enters a string of 30 characters, cin will write past the end of the array. This can be prevented by using cin's getline member function. Assume the following array has been defined in a program:

```
const int SIZE = 80;
char line[SIZE];
```

The following statement uses cin's getline member function to get a line of input (including whitespace characters) and store it in the line array:

```
cin.getline(line, SIZE);
```

The first argument tells getline where to store the string input. This statement indicates the starting address of the line array as the storage location for the string. The second argument indicates the maximum length of the string, including the null terminator. In this example, the SIZE constant is equal to 80, so cin will read 79 characters, or until the user presses the Enter key, whichever comes first. cin will automatically append the null terminator to the end of the string.

Once a string is stored in an array, it can be processed using standard subscript notation. For example, Program 10-5 displays a string stored in an array. It uses a loop to display each character in the array until the null terminator is encountered.

Program 10-5

```
1  // This program displays a string stored in a char array.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      const int SIZE = 80;  // Array size
8      char line[SIZE];      // To hold a line of input
9      int count = 0;        // Loop counter variable
10
11     // Get a line of input.
12     cout << "Enter a sentence of no more than "
13          << (SIZE - 1) << " characters:\n";
14     cin.getline(line, SIZE);
15
```

(program continues)

Program 10-5 *(continued)*

```
16      // Display the input one character at a time.
17      cout << "The sentence you entered is:\n";
18      while (line[count] != '\0')
19      {
20           cout << line[count];
21           count++;
22      }
23      return 0;
24  }
```

Program Output with Example Input Shown in Bold

```
Enter a sentence of no more than 79 characters:
C++ is a powerful programming language! Enter
The sentence you entered is:
C++ is a powerful programming language!
```

10.4 Library Functions for Working with C-Strings

CONCEPT: **The C++ library has numerous functions for handling C-strings. These functions perform various tests and manipulations, and require the <cstring> header file be included.**

The strlen Function

Because C-strings are stored in arrays, working with them is quite different from working with string objects. Fortunately, the C++ library provides many functions for manipulating and testing C-strings. These functions all require the <cstring> header file to be included, as shown here:

```
#include <cstring>
```

For instance, the following code segment uses the strlen function to determine the length of the string stored in the name array:

```
char name[] = "Thomas Edison";
int length;
length = strlen(name);
```

The strlen function accepts a pointer to a C-string as its argument. It returns the length of the string, which is the number of characters up to, but not including, the null terminator. As a result, the variable length will have the number 13 stored in it. The length of a string isn't to be confused with the size of the array holding it. Remember, the only information being passed to strlen is the beginning address of a C-string. It doesn't know where the array ends, so it looks for the null terminator to indicate the end of the string.

When using a C-string-handling function, you must pass one or more C-strings as arguments. This means passing the address of the C-string, which may be accomplished by using any of the following as arguments:

- The name of the array holding the C-string
- A pointer variable that holds the address of the C-string
- A literal string

Anytime a literal string is used as an argument to a function, the address of the literal string is passed. Here is an example of the `strlen` function being used with such an argument:

```
length = strlen("Thomas Edison");
```

The `strcat` Function

The `strcat` function accepts two pointers to C-strings as its arguments. The function *concatenates*, or appends one string to another. The following code shows an example of its use:

```
const int SIZE = 13;
char string1[SIZE] = "Hello ";
char string2[] = "World!";

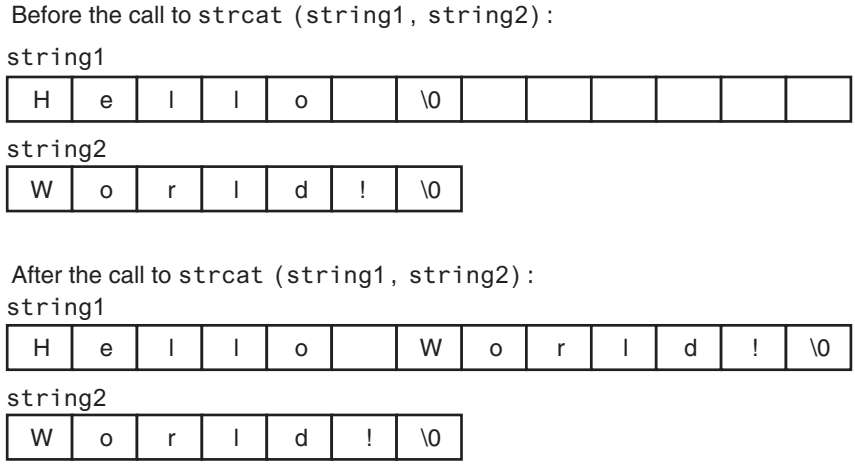
cout << string1 << endl;
cout << string2 << endl;
strcat(string1, string2);
cout << string1 << endl;
```

These statements will cause the following output:

```
Hello
World!
Hello World!
```

The `strcat` function copies the contents of `string2` to the end of `string1`. In this example, `string1` contains the string "Hello " before the call to `strcat`. After the call, it contains the string "Hello World!". Figure 10-2 shows the contents of both arrays before and after the function call.

Figure 10-2 Using `strcat`



Notice the last character in string1 (before the null terminator) is a space. The strcat function doesn't insert a space, so it's the programmer's responsibility to make sure one is already there, if needed. It's also the programmer's responsibility to make sure the array holding string1 is large enough to hold string1 plus string2 plus a null terminator.

Here is a program segment that uses the sizeof operator to test an array's size before strcat is called:

```
if (sizeof(string1) >= (strlen(string1) + strlen(string2) + 1))
    strcat(string1, string2);
else
    cout << "String1 is not large enough for both strings.\n";
```



WARNING! If the array holding the first string isn't large enough to hold both strings, strcat will overflow the boundaries of the array.

The strcpy Function

Recall from Chapter 7 that one array cannot be assigned to another with the = operator. Each individual element must be assigned, usually inside a loop. The strcpy function can be used to copy one string to another. Here is an example of its use:

```
const int SIZE = 13;
char name[SIZE];
strcpy(name, "Albert Einstein");
```

The strcpy function's two arguments are C-string addresses. The contents of the second argument are copied to the memory location specified by the first argument, including the null terminator. (The first argument usually references an array.) In this example, the strcpy function will copy the string "Albert Einstein" to the name array.

If anything is already stored in the location referenced by the first argument, it is overwritten, as shown in the following program segment:

```
const int SIZE = 10;
char string1[SIZE] = "Hello", string2[SIZE] = "World!";
cout << string1 << endl;
cout << string2 << endl;
strcpy(string1, string2);
cout << string1 << endl;
cout << string2 << endl;
```

Here is the output:

```
Hello
World!
World!
World!
```



WARNING! Being true to C++'s nature, strcpy performs no bounds checking. The array specified by the first argument will be overflowed if it isn't large enough to hold the string specified by the second argument.

The strncat and strncpy Functions

Because the strcat and strcpy functions can potentially overwrite the bounds of an array, they make it possible to write unsafe code. As an alternative, you should use strncat and strncpy whenever possible.

The strncat functions works like strcat, except it takes a third argument specifying the maximum number of characters from the second string to append to the first. Here is an example call to strncat:

```
strncat(string1, string2, 10);
```

When this statement executes, strncat will append no more than 10 characters from string2 to string1. The following code shows an example of calculating the maximum number of characters that can be appended to an array:

```
1  int maxChars;
2  const int SIZE_1 = 17;
3  const int SIZE_2 = 18;
4
5  char string1[SIZE_1] = "Welcome ";
6  char string2[SIZE_2] = "to North Carolina";
7
8  cout << string1 << endl;
9  cout << string2 << endl;
10 maxChars = sizeof(string1) - (strlen(string1) + 1);
11 strncat(string1, string2, maxChars);
12 cout << string1 << endl;
```

The statement in line 10 calculates the number of empty elements in string1. It does this by subtracting the length of the string stored in the array plus 1 for the null terminator. This code will cause the following output:

```
Welcome
to North Carolina
Welcome to North
```

The strncpy function allows you to copy a specified number of characters from a string to a destination. Calling strncpy is similar to calling strcpy, except you pass a third argument specifying the maximum number of characters from the second string to copy to the first. Here is an example call to strncpy:

```
strncpy(string1, string2, 5);
```

When this statement executes, strncpy will copy no more than five characters from string2 to string1. However, if the specified number of characters is less than or equal to the length of string2, a null terminator is not appended to string1. If the specified number of characters is greater than the length of string2, then string1 is padded with null terminators, up to the specified number of characters. The following code shows an example using the strncpy function:

```
1  int maxChars;
2  const int SIZE = 11;
3
4  char string1[SIZE];
5  char string2[] = "I love C++ programming!";
6
```

```
 7  maxChars = sizeof(string1) - 1;
 8  strncpy(string1, string2, maxChars);
 9  // Put the null terminator at the end.
10  string1[maxChars] = '\0';
11  cout << string1 << endl;
```

Notice a statement was written in line 10 to put the null terminator at the end of `string1`. This is because `maxChars` was less than the length of `string2`, and `strncpy` did not automatically place a null terminator there.

The `strstr` Function

The `strstr` function searches for a string inside of a string. For instance, it could be used to search for the string "seven" inside the larger string "Four score and seven years ago." The function's first argument is the string to be searched, and the second argument is the string for which to look. If the function finds the second string inside the first, it returns the address of the occurrence of the second string within the first string. Otherwise, it returns `nullptr` (the address 0). Here is an example:

```
char arr[] = "Four score and seven years ago";
char *strPtr = nullptr;
cout << arr << endl;
strPtr = strstr(arr, "seven"); // search for "seven"
cout << strPtr << endl;
```

In this code, `strstr` will locate the string "seven" inside the string "Four score and seven years ago." It will return the address of the first character in "seven" which will be stored in the pointer variable `strPtr`. If run as part of a complete program, this segment will display the following:

```
Four score and seven years ago
seven years ago
```



NOTE: The `nullptr` key word was introduced in C++ 11. If you are using a previous version of the C++ language, use the constant `NULL` instead.

The `strstr` function can be useful in any program that must locate data inside one or more strings. Program 10-6, for example, stores a list of product numbers and descriptions in an array of C-strings. It allows the user to look up a product description by entering all or part of its product number.

Program 10-6

```
 1  // This program uses the strstr function to search an array.
 2  #include <iostream>
 3  #include <cstring>    // For strstr
 4  using namespace std;
 5
 6  int main()
 7  {
 8      // Constants for array lengths
 9      const int NUM_PRODS = 5;    // Number of products
10      const int LENGTH = 27;      // String length
```

```
11
12      // Array of products
13      char products[NUM_PRODS][LENGTH] =
14                      { "TV327 31-inch Television",
15                        "CD257 CD Player",
16                        "TA677 Answering Machine",
17                        "CS109 Car Stereo",
18                        "PC955 Personal Computer" };
19
20      char lookUp[LENGTH];      // To hold user's input
21      char *strPtr = nullptr;   // To point to the found product
22      int index;                // Loop counter
23
24      // Prompt the user for a product number.
25      cout << "\tProduct Database\n\n";
26      cout << "Enter a product number to search for: ";
27      cin.getline(lookUp, LENGTH);
28
29      // Search the array for a matching substring
30      for (index = 0; index < NUM_PRODS; index++)
31      {
32          strPtr = strstr(products[index], lookUp);
33          if (strPtr != nullptr)
34              break;
35      }
36
37      // If a matching substring was found, display the product info.
38      if (strPtr != nullptr)
39          cout << products[index] << endl;
40      else
41          cout << "No matching product was found.\n";
42
43      return 0;
44  }
```

Program Output with Example Input Shown in Bold

```
      Product Database

Enter a product to search for: CS [Enter]
CS109 Car Stereo
```

Program Output with Different Example Input Shown in Bold

```
      Product Database

Enter a product to search for: AB [Enter]
No matching product was found.
```

Table 10-3 summarizes the string-handling functions discussed here, as well as the `strcmp` function that was discussed in Chapter 4. (All the functions listed require the `<cstring>` header file.)

In Program 10-6, the `for` loop in lines 30 through 35 steps through each C-string in the array calling the following statement:

```
strPtr = strstr(prods[index], lookUp);
```

The strstr function searches the string referenced by prods[index] for the name entered by the user, which is stored in lookUp. If lookUp is found inside prods[index], the function returns its address. In that case, the following if statement causes the for loop to terminate:

```
if (strPtr != nullptr)
    break;
```

Outside the loop, the following if else statement in lines 38 through 41 determines whether the string entered by the user was found in the array. If not, it informs the user that no matching product was found. Otherwise, the product number and description are displayed.

```
if (strPtr == nullptr)
    cout << "No matching product was found.\n";
else
    cout << prods[index] << endl;
```

The strcmp Function

Because C-strings are stored in char arrays, you cannot use the relational operators to compare two C-strings. To compare C-strings, you should use the library function strcmp. This function takes two C-strings as arguments and returns an integer that indicates how the two strings compare to each other. Here is the function's prototype:

```
int strcmp(char *string1, char *string2);
```

The function takes two C-strings as parameters (actually, pointers to C-strings) and returns an integer result. The value of the result is set accordingly:

- The result is *zero* if the two strings are *equal* on a character-by-character basis.
- The result is *negative* if string1 comes *before* string2 in alphabetical order.
- The result is *positive* if string1 comes *after* string2 in alphabetical order.

Here is an example of the use of strcmp to determine if two strings are equal:

```
if (strcmp(string1, string2) == 0)
    cout << "The strings are equal.\n";
else
    cout << "The strings are not equal.\n";
```

Program 10-7 shows a complete example.

Program 10-7

```
1  // This program tests two C-strings for equality
2  // using the strcmp function.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main()
8  {
9      // Two arrays for two strings.
10     const int LENGTH = 40;
11     char firstString[LENGTH], secondString[LENGTH];
12
```

```
13      // Read two strings.
14      cout << "Enter a string: ";
15      cin.getline(firstString, LENGTH);
16      cout << "Enter another string: ";
17      cin.getline(secondString, LENGTH);
18
19      // Compare the strings for equality with strcmp.
20      if (strcmp(firstString, secondString) == 0)
21          cout << "You entered the same string twice.\n";
22      else
23          cout << "The strings are not the same.\n";
24
25      return 0;
26  }
```

Program Output with Example Input Shown in Bold

```
Enter a string: Alfonso [Enter]
Enter another string: Alfonso [Enter]
You entered the same string twice.
```

The `strcmp` function is case sensitive when it compares strings. If the user enters "Dog" and "dog" in Program 10-7, it will report they are not the same. Most compilers provide nonstandard versions of `strcmp` that perform case-insensitive comparisons. For instance, some compilers provide a function named `stricmp` that works identically to `strcmp`, except the case of the characters is ignored.

Program 10-8 is a more practical example of how `strcmp` can be used. It asks the user to enter the part number of the MP3 player they wish to purchase. The part number contains digits, letters, and a hyphen, so it must be stored as a string. Once the user enters the part number, the program displays the price of the stereo.

Program 10-8

```
1   // This program uses strcmp to compare the string entered
2   // by the user with the valid MP3 player part numbers.
3   #include <iostream>
4   #include <cstring>
5   #include <iomanip>
6   using namespace std;
7
8   int main()
9   {
10      // Price of parts.
11      const double A_PRICE = 99.0,
12                   B_PRICE = 199.0;
13
14      // Character array for part number.
15      const int PART_LENGTH = 9;
16      char partNum[PART_LENGTH];
17
```

(program continues)

Program 10-8 *(continued)*

```
18      // Instruct the user to enter a part number.
19      cout << "The MP3 player part numbers are:\n"
20           << "\t16 Gigabyte, part number S147-29A\n"
21           << "\t32 Gigabyte, part number S147-29B\n"
22           << "Enter the part number of the MP3 player you\n"
23           << "wish to purchase: ";
24
25      // Read a part number of at most 8 characters.
26      cin >> partNum;
27
28      // Determine what user entered using strcmp
29      // and print its price.
30      cout << showpoint << fixed << setprecision(2);
31      if (strcmp(partNum, "S147-29A") == 0)
32          cout << "The price is $" << A_PRICE << endl;
33      else if (strcmp(partNum, "S147-29B") == 0)
34          cout << "The price is $" << B_PRICE << endl;
35      else
36          cout << partNum << " is not a valid part number.\n";
37      return 0;
38  }
```

Program Output with Example Input Shown in Bold

```
The MP3 player part numbers are:
    16 Gigabyte, part number S147-29A
    32 Gigabyte, part number S147-29B
Enter the part number of the stereo you
wish to purchase: S147-29B [Enter]
The price is $199.00
```

Using ! with strcmp

Some programmers prefer to use the logical NOT operator with `strcmp` when testing strings for equality. Because 0 is considered logically false, the `!` operator converts that value to true. The expression `!strcmp(string1, string2)` returns `true` when both strings are the same, and false when they are different. The two following statements perform the same operation:

```
if (strcmp(firstString, secondString) == 0)
if (!strcmp(firstString, secondString))
```

Sorting Strings

Programs are frequently written to print alphabetically sorted lists of items. For example, consider a department store computer system that keeps customers' names and addresses in a file. The names do not appear in the file alphabetically, but in the order the operator entered them. If a list were to be printed in this order, it would be very difficult to locate any specific name. The list would have to be sorted before it was printed.

Because the value returned by `strcmp` is based on the relative alphabetic order of the two strings being compared, it can be used in programs that sort strings. Program 10-9 asks the user to enter two names, which are then printed in alphabetic order.

Program 10-9

```
// This program uses the return value of strcmp to
// alphabetically sort two strings entered by the user.
#include <iostream>
#include <string>
using namespace std;

int main()
{
      // Two arrays to hold two strings.
      const int NAME_LENGTH = 30;
      char name1[NAME_LENGTH], name2[NAME_LENGTH];

      // Read two strings.
      cout << "Enter a name (last name first): ";
      cin.getline(name1, NAME_LENGTH);
      cout << "Enter another name: ";
      cin.getline(name2, NAME_LENGTH);

      // Print the two strings in alphabetical order.
      cout << "Here are the names sorted alphabetically:\n";
      if (strcmp(name1, name2) < 0)
          cout << name1 << endl << name2 << endl;
      else if (strcmp(name1, name2) > 0)
          cout << name2 << endl << name1 << endl;
      else
          cout << "You entered the same name twice!\n";

      return 0;
}
```

Program Output with Example Input Shown in Bold

```
Enter a name (last name first): Smith, Richard [Enter]
Enter another name: Jones, John [Enter]
Here are the names sorted alphabetically:
Jones, John
Smith, Richard
```

Table 10-3 provides a summary of the C-string-handling functions we have discussed. All of the functions listed require the `<cstring>` header file.

Table 10-3 Some of the C-String Functions in `<cstring>`

Function	Description
`strlen`	Accepts a C-string or a pointer to a C-string as an argument. Returns the length of the C-string (not including the null terminator.) *Example Usage:* `len = strlen(name);`
`strcat`	Accepts two C-strings or pointers to two C-strings as arguments. The function appends the contents of the second string to the first C-string. (The first string is altered, the second string is left unchanged.) *Example Usage:* `strcat(string1, string2);`

(table continues)

Table 10-3 *(continued)*

Function	Description
strcpy	Accepts two C-strings or pointers to two C-strings as arguments. The function copies the second C-string to the first C-string. The second C-string is left unchanged. *Example Usage:* `strcpy(string1, string2);`
strncat	Accepts two C-strings or pointers to two C-strings, and an integer argument. The third argument, an integer, indicates the maximum number of characters to copy from the second C-string to the first C-string. *Example Usage:* `strncat(string1, string2, n);`
strncpy	Accepts two C-strings or pointers to two C-strings, and an integer argument. The third argument, an integer, indicates the maximum number of characters to copy from the second C-string to the first C-string. If `n` is less than the length of `string2`, the null terminator is not automatically appended to `string1`. If `n` is greater than the length of `string2`, `string1` is padded with '\0' characters. *Example Usage:* `strncpy(string1, string2, n);`
strcmp	Accepts two C-strings or pointers to two C-strings arguments. If `string1` and `string2` are the same, this function returns 0. If `string2` is alphabetically greater than `string1`, it returns a negative number. If `string2` is alphabetically less than `string1`, it returns a positive number. *Example Usage:* `if (strcmp(string1, string2))`
strstr	Accepts two C-strings or pointers to two C-strings as arguments. Searches for the first occurrence of `string2` in `string1`. If an occurrence of `string2` is found, the function returns a pointer to it. Otherwise, it returns `nullptr` (address 0). *Example Usage:* `cout << strstr(string1, string2);`



Checkpoint

10.6 Write a short description of each of the following functions:

```
strlen
strcat
strcpy
strncat
strncpy
strcmp
strstr
```

10.7 What will the following program segment display?

```
char dog[] = "Fido";
cout << strlen(dog) << endl;
```

10.8 What will the following program segment display?

```
char string1[16] = "Have a ";
char string2[9] = "nice day";
strcat(string1, string2);
cout << string1 << endl;
cout << string2 << endl;
```

10.9 Write a statement that will copy the string "Beethoven" to the array `composer`.

10.10 When complete, the following program skeleton will search for the string "Windy" in the array `place`. If `place` contains "Windy" the program will display the message "Windy found." Otherwise, it will display "Windy not found."

```
#include <iostream>
// include any other necessary header files
using namespace std;

int main()
{
    char place[] = "The Windy City";
    // Complete the program. It should search the array place
    // for the string "Windy" and display the message "Windy
    // found" if it finds the string. Otherwise, it should
    // display the message "Windy not found."
    return 0;
}
```

10.5 String/Numeric Conversion Functions

CONCEPT: The C++ library provides functions for converting C-strings and `string` objects to numeric data types and vice versa.

There is a great difference between a number that is stored as a string, and one stored as a numeric value. The string "26792" isn't actually a number, but a series of ASCII codes representing the individual digits of the number. It uses 6 bytes of memory (including the null terminator). Because it isn't an actual number, it's not possible to perform mathematical operations with it, unless it is first converted to a numeric value.

Several functions exist in the C++ library for converting C-string representations of numbers into numeric values. Table 10-4 shows some of these. Note all of these functions require the `<cstdlib>` header file.

Table 10-4 C-String/Numeric Conversion Functions in `<cstdlib>`

Function	Description
`atoi`	Accepts a C-string as an argument. The function converts the C-string to an integer and returns that value. *Example Usage:* `int num = atoi("4569");`
`atol`	Accepts a C-string as an argument. The function converts the C-string to a `long` integer and returns that value. *Example Usage:* `long lnum = atol("500000");`
`atof`	Accepts a C-string as an argument. The function converts the C-string to a `double` and returns that value. *Example Usage:* `double fnum = atof("3.14159");`



NOTE: If a C-string that cannot be converted to a numeric value is passed to any of the functions in Table 10-4, the function's behavior is undefined by C++. Many compilers, however, will perform the conversion process until an invalid character is encountered. For example, `atoi("123x5")` might return the integer 123. It is possible that these functions will return 0 if they cannot successfully convert their argument.

The string to Number Functions

C++ 11 introduced several new functions that convert `string` objects to numeric values. Table 10-5 lists the functions. To use any of the functions in Table 10-5, you must `#include` the `<string>` header file.

Table 10-5 `string` to Number Functions

Function	Description
`stoi(string str)`	Accepts a `string` argument and returns that argument's value converted to an `int`.
`stol(string str)`	Accepts a `string` argument and returns that argument's value converted to a `long`.
`stoul(string str)`	Accepts a `string` argument and returns that argument's value converted to an `unsigned long`.
`stoll(string str)`	Accepts a `string` argument and returns that argument's value converted to a `long long`.
`stoull(string str)`	Accepts a `string` argument and returns that argument's value converted to an `unsigned long long`.
`stof(string str)`	Accepts a `string` argument and returns that argument's value converted to a `float`.
`stod(string str)`	Accepts a `string` argument and returns that argument's value converted to a `double`.
`stold(string str)`	Accepts a `string` argument and returns that argument's value converted to a `long double`.



NOTE: If a `string` that cannot be converted to a numeric value is passed to any of the functions in Table 10-5, the function will throw an `invalid_argument` exception. If the `string` argument converts to a value that is out of range for the target data type, the function will throw an `out_of_range` exception.

The functions shown in Table 10-5 can accept either a `string` object or a C-string as an argument. For example, each of the following code snippets use the `stoi` function to convert the string "99" to an `int`:

```
// Convert a string object to an int.
string str = "99";
int i = stoi(str);

// Convert a string literal to an int.
int i = stoi("99");
```

```
// Convert a C-String in a char array to an int.
char cstr[] = "99";
int i = stoi(cstr);
```

The to_string Function

C++ 11 introduced a function named to_string that converts a numeric value to a string object. There are nine overloaded versions of the to_string function listed in Table 10-6. Note the to_string function requires the <string> header file to be included.

Each version of the function accepts an argument of a numeric data type and returns the value of that argument converted to a string object. Here is an example:

```
int number = 99;
string output = to_string(number);
```

Table 10-6 Overloaded Versions of the to_string Function

Function	Description
to_string(int *value*);	Accepts an int argument and returns that argument converted to a string object.
to_string(long *value*);	Accepts a long argument and returns that argument converted to a string object.
to_string(long long *value*);	Accepts a long long argument and returns that argument converted to a string object.
to_string(unsigned *value*);	Accepts an unsigned argument and returns that argument converted to a string object.
to_string(unsigned long *value*);	Accepts an unsigned long argument and returns that argument converted to a string object.
to_string(unsigned long long *value*);	Accepts an unsigned long long argument and returns that argument converted to a string object.
to_string(float *value*);	Accepts a float argument and returns that argument converted to a string object.
to_string(double *value*);	Accepts a double argument and returns that argument converted to a string object.
to_string(long double *value*);	Accepts a long double argument and returns that argument converted to a string object.

The first statement initializes the number variable (an int) to the value 99. In the second statement, the number variable is passed as an argument to the to_string function. The to_string function returns the value "99", which is assigned to the output variable.

Here is another example:

```
double number = 3.14159;
cout << to_string(number) << endl;
```

The first statement initializes the number variable (a double) to the value 3.14159. In the second statement, the number variable is passed as an argument to the to_string function, and the value "3.14159" is returned from the function and displayed on the screen.

Now let's look at Program 10-10, which uses one of the C++ 11 string-to-number conversion functions, stoi. It allows the user to enter a series of values, or the letter Q or q to quit. The average of the numbers is then calculated and displayed.

Program 10-10

```
// This program demonstrates the tolower and stoi functions.
#include <iostream>
#include <cctype>         // For tolower
#include <string>
using namespace std;

int main()
{
   string input;          // To hold user input
   int total = 0;         // Accumulator
   int count = 0;         // Loop counter
   double average;        // To hold the average of numbers

   // Get the first number.
   cout << "This program will average a series of numbers.\n";
   cout << "Enter the first number or Q to quit: ";
   getline(cin, input);

   // Process the number and subsequent numbers.
   while (tolower(input[0]) != 'q')
   {
      total += stoi(input);     // Keep a running total
      count++;                  // Count the numbers entered
      // Get the next number.
      cout << "Enter the next number or Q to quit: ";
      getline(cin, input);
   }

   // If any numbers were entered, display their average.
   if (count != 0)
   {
      average = static_cast<double>(total) / count;
      cout << "Average: " << average << endl;
   }
   return 0;
}
```

Program Output with Example Input Shown in Bold

```
This program will average a series of numbers.
Enter the first number or Q to quit: 1 [Enter]
Enter the next number or Q to quit: 2 [Enter]
Enter the next number or Q to quit: 3 [Enter]
Enter the next number or Q to quit: 4 [Enter]
Enter the next number or Q to quit: 5 [Enter]
Enter the next number or Q to quit: q [Enter]
Average: 3
```

In line 20, the following `while` statement uses the `tolower` function to determine whether the first character entered by the user is "`q`" or "`Q`".

```
while (tolower(input[0]) != 'q')
```

If the user hasn't entered 'Q' or 'q', the loop performs an iteration. The following statement, in line 22, uses `stoi` to convert `input` to an integer, and adds its value to `total`:

```
total += stoi(input);    // Keep a running total
```

The counter is updated in line 23, then the user is asked for the next number. When all the numbers are entered, the user terminates the loop by entering '`Q`' or '`q`'. If one or more numbers are entered, their average is displayed.

The string-to-numeric conversion functions can also help with a common input problem. Recall from Chapter 3 that using `cin >>` then calling `cin.get` causes problems because the `>>` operator leaves the newline character in the keyboard buffer. When the `cin.get` function executes, the first character it sees in the keyboard buffer is the newline character, so it reads no further.

The same problem exists when a program uses `cin >>` then calls `getline` to read a line of input. For example, look at the following code. (Assume `idNumber` is an `int` and `name` is a `string` object.)

```
1  // Get the user's ID number.
2  cout << "What is your ID number? ";
3  cin >> idNumber;
4
5  // Get the user's name.
6  cout << "What is your name? ";
7  getline(cin, name);
```

Let's say the user enters 25 and presses Enter when the `cin >>` statement in line 3 executes. The value 25 will be stored in `idNumber`, and the newline character will be left in the keyboard buffer. When the `getline` function is called in line 7, the first character it sees in the keyboard buffer is the newline character, so it reads no further. It will appear that the statement in line 7 was skipped.

One work-around that we have used in this book is to call `cin.ignore` to skip over the newline character just before calling `getline`. Another approach is to use `getline` to read *all* of a program's input, including numbers. When numeric input is needed, it is read into a `string` object as a string then converted to the appropriate numeric data type. Because you aren't mixing `cin >>` with `getline`, the problem of the remaining newline character doesn't exist. Program 10-11 shows an example.

Program 10-11

```
1  // This program demonstrates how the getline function can
2  // be used for all of a program's input.
3  #include <iostream>
4  #include <string>
5  #include <iomanip>
6  using namespace std;
7
```

(program continues)

Program 10-11 *(continued)*

```
 8  int main()
 9  {
10     string input;        // To hold a line of input
11     string name;         // To hold a name
12     int idNumber;        // To hold an ID number.
13     int age;             // To hold an age
14     double income;       // To hold income
15
16     // Get the user's ID number.
17     cout << "What is your ID number? ";
18     getline(cin, input);             // Read as a string
19     idNumber = stoi(input);          // Convert to int
20
21     // Get the user's name. No conversion necessary.
22     cout << "What is your name? ";
23     getline(cin, name);
24
25     // Get the user's age.
26     cout << "How old are you? ";
27     getline(cin, input);             // Read as a string
28     age = stoi(input);               // Convert to int
29
30     // Get the user's income.
31     cout << "What is your annual income? ";
32     getline(cin, input);             // Read as a string
33     income = stod(input);            // Convert to double
34
35     // Show the resulting data.
36     cout << setprecision(2) << fixed << showpoint;
37     cout << "Your name is " << name
38          <<", you are " << age
39          << " years old,\nand you make $"
40          << income << " per year.\n";
41
42     return 0;
43  }
```

Program Output with Example Input Shown in Bold

```
What is your ID number? 1234 [Enter]
What is your name? Janice Smith [Enter]
How old are you? 25 [Enter]
What is your annual income? 60000 [Enter]
Your name is Janice Smith, you are 25 years old,
and you make $60000.00 per year.
```

Checkpoint

10.11 Write a short description of each of the following functions:

```
atoi        stoi
atol        stol
atof        stof
itoa        stod
```

10.12 Write a statement that will convert the string "10" to an integer and store the result in the variable `num`.

10.13 Write a statement that will convert the string "100000" to a `long` and store the result in the variable `num`.

10.14 Write a statement that will convert the string "7.2389" to a `double` and store the result in the variable `num`.

10.15 Write a statement that will convert the integer 127 to a string, and assign the result to a `string` object named `str`.

10.6 Focus on Software Engineering: Writing Your Own C-String-Handling Functions

CONCEPT: **You can design your own specialized functions for manipulating strings.**

VideoNote
Writing a C-String-Handling Function

By being able to pass arrays as arguments, you can write your own functions for processing C-strings. For example, Program 10-12 uses a function to copy a C-string from one array to another.

Program 10-12

```
1  // This program uses a function to copy a C-string into an array.
2  #include <iostream>
3  using namespace std;
4
5  void stringCopy(char [], char []); // Function prototype
6
7  int main()
8  {
9      const int LENGTH = 30;  // Size of the arrays
10     char first[LENGTH];     // To hold the user's input
11     char second[LENGTH];    // To hold the copy
12
13     // Get a string from the user and store in first.
14     cout << "Enter a string with no more than "
15          << (LENGTH - 1) << " characters:\n";
16     cin.getline(first, LENGTH);
17
```

(program continues)

Program 10-12 *(continued)*

```
18       // Copy the contents of first to second.
19       stringCopy(first, second);
20
21       // Display the copy.
22       cout << "The string you entered is:\n" << second << endl;
23       return 0;
24   }
25
26   //*********************************************************
27   // Definition of the stringCopy function.                 *
28   // This function copies the C-string in string1 to string2.*
29   //*********************************************************
30
31   void stringCopy(char string1[], char string2[])
32   {
33       int index = 0;  // Loop counter
34
35       // Step through string1, copying each element to
36       // string2. Stop when the null character is encountered.
37       while (string1[index] != '\0')
38       {
39           string2[index] = string1[index];
40           index++;
41       }
42
43       // Place a null character in string2.
44       string2[index] = '\0';
45   }
```

Program Output with Example Input Shown in Bold

```
Enter a string with no more than 29 characters:
Thank goodness it's Friday! Enter
The string you entered is:
Thank goodness it's Friday!
```

Notice the function `stringCopy` does not accept an argument indicating the size of the arrays. It simply copies the characters from `string1` into `string2` until it encounters a null terminator in `string1`. When the null terminator is found, the loop has reached the end of the C-string. The last statement in the function assigns a null terminator (the `'\0'` character) to the end of `string2`, so it is properly terminated.



WARNING! Because the `stringCopy` function doesn't know the size of the second array, it's the programmer's responsibility to make sure the second array is large enough to hold the string in the first array.

Program 10-13 uses another C-string-handling function: `nameSlice`. The program asks the user to enter his or her first and last names, separated by a space. The function searches the string for the space and replaces it with a null terminator. In effect, this "cuts" the last name off of the string.

Program 10-13

```
 1  // This program uses the function nameSlice to cut the last
 2  // name off of a string that contains the user's first and
 3  // last names.
 4  #include <iostream>
 5  using namespace std;
 6
 7  void nameSlice(char []);  // Function prototype
 8
 9  int main()
10  {
11      const int SIZE = 41; // Array size
12      char name[SIZE];     // To hold the user's name
13
14      cout << "Enter your first and last names, separated ";
15      cout << "by a space:\n";
16      cin.getline(name, SIZE);
17      nameSlice(name);
18      cout << "Your first name is: " << name << endl;
19      return 0;
20  }
21
22  //*************************************************************
23  // Definition of function nameSlice. This function accepts a   *
24  // character array as its argument. It scans the array looking *
25  // for a space. When it finds one, it replaces it with a null  *
26  // terminator.                                                 *
27  //*************************************************************
28
29  void nameSlice(char userName[])
30  {
31      int count = 0; // Loop counter
32
33      // Locate the first space, or the null terminator if there
34      // are no spaces.
35      while (userName[count] != ' ' && userName[count] != '\0')
36          count++;
37
38      // If a space was found, replace it with a null terminator.
39      if (userName[count] == ' ')
40          userName[count] = '\0';
41  }
```

Program Output with Example Input Shown in Bold

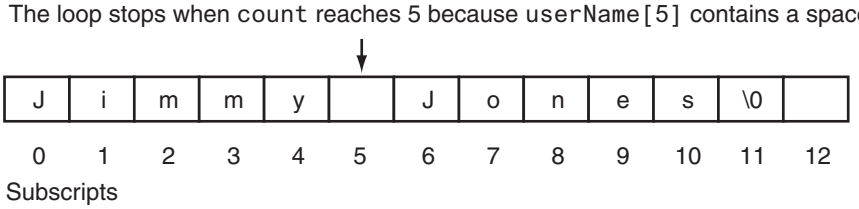
```
Enter your first and last names, separated by a space:
Jimmy Jones [Enter]
Your first name is: Jimmy
```

The following loop in lines 35 and 36 starts at the first character in the array and scans the string searching for either a space or a null terminator:

```
while (userName[count] != ' ' && userName[count] != '\0')
   count++;
```

If the character in `userName[count]` isn't a space or the null terminator, `count` is incremented, and the next character is examined. With the example input "`Jimmy Jones`," the loop finds the space separating "`Jimmy`" and "`Jones`" at `userName[5]`. When the loop stops, `count` is set to 5. This is illustrated in Figure 10-3.

Figure 10-3 Scanning for a space in a string





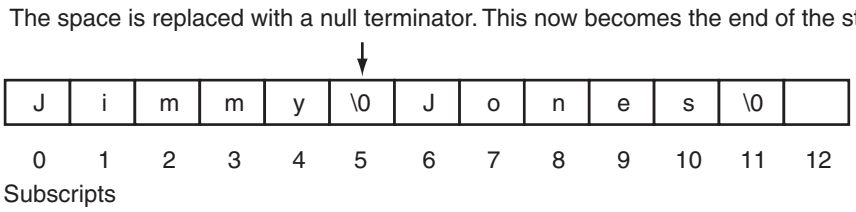
NOTE: The loop will also stop if it encounters a null terminator. This is so it will not go beyond the boundary of the array if the user didn't enter a space.

Once the loop has finished, `userName[count]` will either contain a space or a null terminator. If it contains a space, the following `if` statement (in lines 39 and 40) replaces it with a null terminator:

```
if (userName[count] == ' ')
   userName[count] = '\0';
```

This is illustrated in Figure 10-4.

Figure 10-4 Replacing a space with a null terminator



The new null terminator now becomes the end of the string.

Using Pointers to Pass C-String Arguments

Pointers are extremely useful for writing functions that process C-strings. If the starting address of a string is passed into a pointer parameter variable, it can be assumed that all the characters, from that address up to the byte that holds the null terminator, are part of the string. (It isn't necessary to know the length of the array that holds the string.)

Program 10-14 demonstrates a function, `countChars`, that uses a pointer to count the number of times a specific character appears in a C-string.

Program 10-14

```
1  // This program demonstrates a function, countChars, that counts
2  // the number of times a specific character appears in a string.
3  #include <iostream>
```

```
using namespace std;

int countChars(char *, char); // Function prototype

int main()
{
    const int SIZE = 51;     // Array size
    char userString[SIZE];   // To hold a string
    char letter;             // The character to count

    // Get a string from the user.
    cout << "Enter a string (up to 50 characters): ";
    cin.getline(userString, SIZE);

    // Choose a character whose occurrences within the string will be counted.
    cout << "Enter a character and I will tell you how many\n";
    cout << "times it appears in the string: ";
    cin >> letter;

    // Display the number of times the character appears.
    cout << letter << " appears ";
    cout << countChars(userString, letter) << " times.\n";
    return 0;
}

//*************************************************************
// Definition of countChars. The parameter strPtr is a pointer    *
// that points to a string. The parameter Ch is a character that  *
// the function searches for in the string. The function returns  *
// the number of times the character appears in the string.       *
//*************************************************************

int countChars(char *strPtr, char ch)
{
    int times = 0;  // Number of times ch appears in the string

    // Step through the string counting occurrences of ch.
    while (*strPtr != '\0')
    {
        if (*strPtr == ch)      // If the current character equals ch...
            times++;            // ... increment the counter
        strPtr++;               // Go to the next char in the string.
    }

    return times;
}
```

Program Output with Example Input Shown in Bold

```
Enter a string (up to 50 characters): Starting Out with C++ [Enter]
Enter a character and I will tell you how many
times it appears in the string: t [Enter]
t appears 4 times.
```

In the function countChars, strPtr points to the C-string that is to be searched, and ch contains the character for which to look. The while loop in lines 41 through 46 repeats as long as the character that strPtr points to is not the null terminator:

```
while (*strPtr != '\0')
```

Inside the loop, the if statement in line 43 compares the character that strPtr points to with the character in ch:

```
if (*strPtr == ch)
```

If the two are equal, the variable times is incremented in line 44. (times keeps a running total of the number of times the character appears.) The last statement in the loop is

```
strPtr++;
```

This statement increments the address in strPtr. This causes strPtr to point to the next character in the string. Then, the loop starts over. When strPtr finally reaches the null terminator, the loop terminates, and the function returns the value in times.



Checkpoint

10.16 What is the output of the following program?

```
#include <iostream>
using namespace std;

// Function Prototype
void mess(char []);

int main()
{
   char stuff[] = "Tom Talbert Tried Trains";
   cout << stuff << endl;
   mess(stuff);
   cout << stuff << endl;
   return 0;
}

// Definition of function mess
void mess(char str[])
{
   int step = 0;

   while (str[step] != '\0')
   {
       if (str[step] == 'T')
          str[step] = 'D';
       step++;
   }
}
```

10.7 More about the C++ string Class

CONCEPT: **Standard C++ provides a special data type for storing and working with strings.**



VideoNote
More about the string Class

The string class is an abstract data type. This means it is not a built-in, primitive data type like int or char. Instead, it is a programmer-defined data type that accompanies the C++ language. It provides many capabilities that make storing and working with strings easy and intuitive.

Using the string Class

The first step in using the string class is to #include the <string> header file. This is accomplished with the following preprocessor directive:

```
#include <string>
```

Now you are ready to define a string object. Defining a string object is similar to defining a variable of a primitive type. For example, the following statement defines a string object named movieTitle:

```
string movieTitle;
```

You assign a string value to the movieTitle object with the assignment operator, as shown in the following statement:

```
movieTitle = "Wheels of Fury";
```

The contents of movieTitle are displayed on the screen with the cout object, as shown in the next statement:

```
cout << "My favorite movie is " << movieTitle << endl;
```

Program 10-15 is a complete program that demonstrates the statements shown above.

Program 10-15

```
1  // This program demonstrates the string class.
2  #include <iostream>
3  #include <string>    // Required for the string class.
4  using namespace std;
5
6  int main()
7  {
8      string movieTitle;
9
10     movieTitle = "Wheels of Fury";
11     cout << "My favorite movie is " << movieTitle << endl;
12     return 0;
13 }
```

Program Output

```
My favorite movie is Wheels of Fury
```

As you can see, working with `string` objects is similar to working with variables of other types. For example, Program 10-16 demonstrates how you can use `cin` to read a value from the keyboard into a `string` object.

Program 10-16

```
1  // This program demonstrates how cin can read a string into
2  // a string class object.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main()
8  {
9      string name;
10
11     cout << "What is your name? ";
12     cin >> name;
13     cout << "Good morning " << name << endl;
14     return 0;
15 }
```

Program Output with Example Input Shown in Bold

```
What is your name? Peggy [Enter]
Good morning Peggy
```

Reading a Line of Input into a `string` Object

If you want to read a line of input (with spaces) into a `string` object, use the `getline()` function. Here is an example:

```
string name;
cout << "What is your name? ";
getline(cin, name);
```

The `getline()` function's first argument is the name of a stream object from which you wish to read the input. The function call above passes the `cin` object to `getline()`, so the function reads a line of input from the keyboard. The second argument is the name of a `string` object. This is where `getline()` stores the input that it reads.

Comparing and Sorting `string` Objects

There is no need to use a function such as `strcmp` to compare `string` objects. You may use the `<`, `>`, `<=`, `>=`, `==`, and `!=` relational operators. For example, assume the following definitions exist in a program:

```
string set1 = "ABC";
string set2 = "XYZ";
```

The object `set1` is considered less than the object `set2` because the characters "ABC" alphabetically precede the characters "XYZ." So, the following `if` statement will cause the message "set1 is less than set2" to be displayed on the screen:

```
if (set1 < set2)
   cout << "set1 is less than set2.\n";
```

Relational operators perform comparisons on `string` objects in a fashion similar to the way the `strcmp` function compares C-strings. One by one, each character in the first operand is compared with the character in the corresponding position in the second operand. If all the characters in both strings match, the two strings are equal. Other relationships can be determined if two characters in corresponding positions do not match. The first operand is less than the second operand if the mismatched character in the first operand is less than its counterpart in the second operand. Likewise, the first operand is greater than the second operand if the mismatched character in the first operand is greater than its counterpart in the second operand.

For example, assume a program has the following definitions:

```
string name1 = "Mary";
string name2 = "Mark";
```

The value in `name1`, "Mary," is greater than the value in `name2`, "Mark." This is because the "y" in "Mary" has a greater ASCII value than the "k" in "Mark."

`string` objects can also be compared to C-strings with relational operators. Assuming `str` is a `string` object, all of the following are valid relational expressions:

```
str > "Joseph"
"Kimberly" < str
str == "William"
```

Program 10-17 demonstrates `string` objects and relational operators.

Program 10-17

```
1  // This program uses the == operator to compare a string entered
2  // by the user with the valid part numbers.
3  #include <iostream>
4  #include <iomanip>
5  #include <string>
6  using namespace std;
7
8  int main()
9  {
10    const double APRICE = 249.0;    // Price for part A
11    const double BPRICE = 199.0;    // Price for part B
12    string partNum;                 // Part mumber
13
14    cout << "The headphone part numbers are:\n";
15    cout << "\tNoise canceling, part number S147-29A\n";
16    cout << "\tWireless, part number S147-29B\n";
17    cout << "Enter the part number of the desired headphones: ";
18    cin >> partNum;
19    cout << fixed << showpoint << setprecision(2);
20
21    if (partNum == "S147-29A")
22       cout << "The price is $" << APRICE << endl;
23    else if (partNum == "S147-29B")
24       cout << "The price is $" << BPRICE << endl;
25    else
26       cout << partNum << " is not a valid part number.\n";
```

(program continues)

Program 10-17 *(continued)*

```
27      return 0;
28  }
```

Program Output with Example Input Shown in Bold

```
The headphone part numbers are:
        Noise canceling, part number S147-29A
        Wireless, part number S147-29B
Enter the part number of the desired headphones: S147-29B [Enter]
The price is $199.00
```

You may also use relational operators to sort `string` objects. Program 10-18 demonstrates this.

Program 10-18

```
 1  // This program uses relational operators to alphabetically
 2  // sort two strings entered by the user.
 3  #include <iostream>
 4  #include <string>
 5  using namespace std;
 6
 7  int main ()
 8  {
 9      string name1, name2;
10
11      // Get a name.
12      cout << "Enter a name (last name first): ";
13      getline(cin, name1);
14
15      // Get another name.
16      cout << "Enter another name: ";
17      getline(cin, name2);
18
19      // Display them in alphabetical order.
20      cout << "Here are the names sorted alphabetically:\n";
21      if (name1 < name2)
22          cout << name1 << endl << name2 << endl;
23      else if (name1 > name2)
24          cout << name2 << endl << name1 << endl;
25      else
26          cout << "You entered the same name twice!\n";
27      return 0;
28  }
```

Program Output with Example Input Shown in Bold

```
Enter a name (last name first): Smith, Richard [Enter]
Enter another name: Jones, John [Enter]
Here are the names sorted alphabetically:
Jones, John
Smith, Richard
```

Other Ways to Define string Objects

There are a variety of ways to initialize a string object when you define it. Table 10-7 shows several example definitions and describes each. Program 10-19 demonstrates a string object initialized with the string "William Smith."

Program 10-19

```
1  // This program initializes a string object.
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  int main()
7  {
8       string greeting;
9       string name("William Smith");
10
11      greeting = "Hello ";
12      cout << greeting << name << endl;
13      return 0;
14 }
```

Program Output

```
Hello William Smith
```

Table 10-7 Examples of string Object Definitions

Definition	Description
`string address;`	Defines an empty string object named address.
`string name("William Smith");`	Defines a string object named name, initialized with "William Smith."
`string person1(person2);`	Defines a string object named person1, which is a copy of person2. person2 may be either a string object or character array.
`string str1(str2, 5);`	Defines a string object named str1, which is initialized to the first five characters in the character array str2.
`string lineFull('z', 10);`	Defines a string object named lineFull initialized with 10 'z' characters.
`string firstName(fullName, 0, 7);`	Defines a string object named firstName, initialized with a substring of the string fullName. The substring is seven characters long, beginning at position 0.

Notice in Program 10-19 the use of the = operator to assign a value to the string object (line 11). The string class supports several operators, which are described in Table 10-8.

Table 10-8 string Class Operators

Supported Operator	Description
>>	Extracts characters from a stream and inserts them into the string. Characters are copied until a whitespace or the end of the string is encountered.
<<	Inserts the string into a stream.
=	Assigns the string on the right to the string object on the left.
+=	Appends a copy of the string on the right to the string object on the left.
+	Returns a string that is the concatenation of the two string operands.
[]	Implements array-subscript notation, as in name[x]. A reference to the character in the x position is returned.
Relational Operators	Each of the relational operators is implemented: < > <= >= == ! =

Program 10-20 demonstrates some of the string operators.

Program 10-20

```
// This program demonstrates the C++ string class.
#include <iostream>
#include <string>
using namespace std;

int main ()
{
     // Define three string objects.
     string str1, str2, str3;

     // Assign values to all three.
     str1 = "ABC";
     str2 = "DEF";
     str3 = str1 + str2;

     // Display all three.
     cout << str1 << endl;
     cout << str2 << endl;
     cout << str3 << endl;

     // Concatenate a string onto str3 and display it.
     str3 += "GHI";
     cout << str3 << endl;
     return 0;
}
```

Program Output

```
ABC
DEF
ABCDEF
ABCDEFGHI
```

Using auto to Define a string Object

Recall from Chapter 2 that you can use the auto key word to define a variable, as long as you provide an initialization value. When using the auto key word, the compiler determines the data type of the variable from the initialization value. Here is an example of using the auto key word to define a variable with a string literal as the initialization value:

```
auto str = "Hello World";
```

The string literal "Hello World" is stored in memory as a C-string, and the compiler will determine its data type to be either const char * (pointer to const char), or const char[*n*] (where *n* is the number of characters in the string, plus one for the null terminator). So, the data type of the str variable will be one of these.

If you want to use the auto key word to define a string object, you can append the s suffix to the string literal, as shown here:

```
auto str = "Hello World"s;
```

The s suffix causes the string "Hello World" to be stored in memory as a string object. As a result, the data type of str will be string.

Using string Class Member Functions

The string class also has member functions. For example, the length member function returns the length of the string stored in the object. The value is returned as an unsigned integer.

Assume the following string object definition exists in a program:

```
string town = "Charleston";
```

The following statement in the same program would assign the value 10 to the variable x.

```
x = town.length();
```

Program 10-21 further demonstrates the length member function.

Program 10-21

```
1  // This program demonstrates a string
2  // object's length member function.
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  int main ()
8  {
9      string town;
10
11     cout << "Where do you live? ";
12     cin >> town;
13     cout << "Your town's name has " << town.length() ;
```

(program continues)

Program 10-21 *(continued)*

```
14       cout << " characters\n";
15       return 0;
16   }
```

Program Output with Example Input Shown in Bold

```
Where do you live? Jacksonville [Enter]
Your town's name has 12 characters
```

The `size` function also returns the length of the string. It is demonstrated in the `for` loop in Program 10-22.

Program 10-22

```
 1  // This program demonstrates the C++ string class.
 2  #include <iostream>
 3  #include <string>
 4  using namespace std;
 5
 6  int main()
 7  {
 8       // Define three string objects.
 9       string str1, str2, str3;
10
11       // Assign values to all three.
12       str1 = "ABC";
13       str2 = "DEF";
14       str3 = str1 + str2;
15
16       // Use subscripts to display str3 one character
17       // at a time.
18       for (int x = 0; x < str3.size(); x++)
19            cout << str3[x];
20       cout << endl;
21
22       // Compare str1 with str2.
23       if (str1 < str2)
24            cout << "str1 is less than str2\n";
25       else
26            cout << "str1 is not less than str2\n";
27       return 0;
28  }
```

Program Output

```
ABCDEF
str1 is less than str2
```

Table 10-9 lists many of the `string` class member functions and their overloaded variations. In the examples, assume `mystring` is the name of a `string` object.

Table 10-9 string Class Member Functions

Member Function Example	Description
`mystring.append(n, 'z')`	Appends `n` copies of `'z'` to `mystring`.
`mystring.append(str)`	Appends `str` to `mystring`. `str` can be a `string` object or character array.
`mystring.append(str, n)`	The first `n` characters of the character array `str` are appended to `mystring`.
`mystring.append(str, x, n)`	`n` number of characters from `str`, starting at position `x`, are appended to `mystring`. If `mystring` is too small, the function will copy as many characters as possible.
`mystring.assign(n, 'z')`	Assigns `n` copies of `'z'` to `mystring`.
`mystring.assign(str)`	Assigns `str` to `mystring`. `str` can be a `string` object or character array.
`mystring.assign(str, n)`	The first `n` characters of the character array `str` are assigned to `mystring`.
`mystring.assign(str, x, n)`	`n` number of characters from `str`, starting at position `x`, are assigned to `mystring`. If `mystring` is too small, the function will copy as many characters as possible.
`mystring.at(x)`	Returns the character at position `x` in the `string`.
`mystring.back()`	Returns the last character in the `string`. (This member function was introduced in C++ 11.)
`mystring.begin()`	Returns an iterator pointing to the first character in the `string`. (For more information on iterators, see Chapter 16.)
`mystring.c_str()`	Converts the contents of `mystring` to a C-string, and returns a pointer to the C-string.
`mystring.capacity()`	Returns the size of the storage allocated for the `string`.
`mystring.clear()`	Clears the `string` by deleting all the characters stored in it.
`mystring.compare(str)`	Performs a comparison like the `strcmp` function (see Chapter 4), with the same return values. `str` can be a `string` object or a character array.
`mystring.compare(x, n, str)`	Compares `mystring` and `str`, starting at position `x`, and continuing for `n` characters. The return value is like `strcmp`. `str` can be a `string` object or character array.
`mystring.copy(str, x, n)`	Copies the character array `str` to `mystring`, beginning at position `x`, for `n` characters. If `mystring` is too small, the function will copy as many characters as possible.
`mystring.empty()`	Returns `true` if `mystring` is empty.
`mystring.end()`	Returns an iterator pointing to the last character of the string in `mystring`. (For more information on iterators, see Chapter 17.)
`mystring.erase(x, n)`	Erases `n` characters from `mystring`, beginning at position `x`.
`mystring.find(str, x)`	Returns the first position at or beyond position `x` where the string `str` is found in `mystring`. `str` may be either a `string` object or a character array.

(table continues)

Table 10-9 *(continued)*

Member Function Example	Description
`mystring.find('z', x)`	Returns the first position at or beyond position `x` where `'z'` is found in `mystring`. If `'z'` is not found, the function returns the special value `string::npos`.
`mystring.front()`	Returns the first character in the `string`. (This member function was introduced in C++ 11.)
`mystring.insert(x, n, 'z')`	Inserts `'z'` `n` times into `mystring` at position `x`.
`mystring.insert(x, str)`	Inserts a copy of `str` into `mystring`, beginning at position `x`. `str` may be either a `string` object or a character array.
`mystring.length()`	Returns the length of the string in `mystring`.
`mystring.replace(x, n, str)`	Replaces the `n` characters in `mystring` beginning at position `x` with the characters in string object `str`.
`mystring.resize(n, 'z')`	Changes the size of the allocation in `mystring` to `n`. If `n` is less than the current size of the string, the string is truncated to `n` characters. If `n` is greater, the string is expanded and `'z'` is appended at the end enough times to fill the new spaces.
`mystring.size()`	Returns the length of the string in `mystring`.
`mystring.substr(x, n)`	Returns a copy of a substring. The substring is `n` characters long and begins at position `x` of `mystring`.
`mystring.swap(str)`	Swaps the contents of `mystring` with `str`.



In the Spotlight:

String Tokenizing

Sometimes a string will contain a series of words or other items of data separated by spaces or other characters. For example, look at the following string:

```
"peach raspberry strawberry vanilla"
```

This string contains the following four items of data: `peach`, `raspberry`, `strawberry`, and `vanilla`. In programming terms, items such as these are known as *tokens*. Notice a space appears between the items. The character that separates tokens is known as a *delimiter*. Here is another example:

```
"17;92;81;12;46;5"
```

This string contains the following tokens: `17`, `92`, `81`, `12`, `46`, and `5`. Notice a semicolon appears between each item. In this example, the semicolon is used as a delimiter. Some programming problems require you to read a string that contains a list of items then extract all of the tokens from the string for processing. For example, look at the following string that contains a date:

```
"3–22–2020"
```

The tokens in this string are 3, 22, and 2020, and the delimiter is the hyphen character. Perhaps a program needs to extract the month, day, and year from such a string. Another example is an operating system pathname, such as the following:

```
/home/rsullivan/data
```

The tokens in this string are home, rsullivan, and data, and the delimiter is the '/' character. Perhaps a program needs to extract all of the directory names from such a pathname. The process of breaking a string into tokens is known as *tokenizing*, or *splitting* a string.

The following function, split, shows an example of how we can split a string into tokens. The function has three parameters:

- s—the string that we want to split into tokens
- delim—the character that is used as a delimiter
- tokens—a vector that will hold the tokens once they are extracted

```
1  void split(const string& s, char delim, vector<string>& tokens)
2  {
3     int tokenStart = 0;  // Starting position of the next token
4
5     // Find the first occurrence of the delimiter.
6     int delimPosition = s.find(delim);
7
8     // While we haven't run out of delimiters...
9     while (delimPosition != string::npos)
10    {
11       // Extract the token.
12       string tok = s.substr(tokenStart, delimPosition - tokenStart);
13
14       // Push the token onto the tokens vector.
15       tokens.push_back(tok);
16
17       // Move delimPosition to the next character position.
18       delimPosition++;
19
20       // Move tokenStart to delmiPosition.
21       tokenStart = delimPosition;
22
23       // Find the next occurrence of the delimiter.
24       delimPosition = s.find(delim, delimPosition);
25
26       // If no more delimiters, extract the last token.
27       if (delimPosition == string::npos)
28       {
29          // Extract the token.
30          string tok = s.substr(tokenStart, delimPosition - tokenStart);
31
32          // Push the token onto the vector.
33          tokens.push_back(tok);
34       }
35    }
36 }
```

Let's take a closer look at the code:

- The `tokenStart` variable defined in line 3 will be used to hold the starting position of the next token. This variable is initialized with 0, assuming the first token starts at position 0.
- In line 6, we call the `s.find()` function, passing `delim` as an argument. The function will return the position of the first occurrence of `delim` in `s`. That value is assigned to the `delimPosition` variable. Note if the `find()` member function does not find the specified delimiter, it will return the special constant `string::npos` (which is defined as –1).
- The `while` loop that begins in line 9 iterates as long as `delimPosition` is not equal to `string::npos`.
- Inside the `while` loop, line 12 extracts the substring beginning at `tokenStart`, and continuing up to the delimiter. The substring, which is a token, is assigned to the `tok` object.
- In line 15, we push the `tok` object to the back of the `tokens` vector.
- Now we are ready to find the next token. Line 18 increments `delimPosition`, and line 21 sets `tokenStart` to the same value as `delimPosition`.
- Line 24 calls the `s.find()` function, passing `delim` and `delimPosition` as arguments. The function will return the position of the next occurrence of `delim`, appearing at or after `delimPosition`. That value is assigned to the `delimPosition` variable. If the specified delimiter is not found, the `find()` member function will return the constant `string::npos`. When that happens in this statement, it means we are processing the last token in the string.
- The `if` statement that starts in line 27 determines whether `delimPosition` is equal to `string::npos`. As previously stated, if this is true, it means we are processing the last token. If that is the case, line 30 extracts the token and assigns it to `tok`, and line 33 pushes `tok` to the back of the `tokens` vector.

Program 10-23 demonstrates the `split` function.

Program 10-23

```
 1  #include <iostream>
 2  #include <string>
 3  #include <vector>
 4  using namespace std;
 5
 6  // Function prototype
 7  void split(const string&, char, vector<string>&);
 8
 9  int main()
10  {
11     // Strings to tokenize
12     string str1 = "one two three four";
13     string str2 = "10:20:30:40:50";
14     string str3 = "a/b/c/d/e/f";
15
16     // vector to hold the tokens.
17     vector<string> tokens;
```

```
18
19      // Tokenize str1, using ' ' as the delimiter.
20      split(str1, ' ', tokens);
21      for (auto e : tokens)
22         cout << e << " ";
23      cout << endl;
24      tokens.clear();
25
26      // Tokenize str2, using ':' as the delimiter.
27      split(str2, ':', tokens);
28      for (auto e : tokens)
29         cout << e << " ";
30      cout << endl;
31      tokens.clear();
32
33      // Tokenize str3, using '/' as the delimiter.
34      split(str3, '/', tokens);
35      for (auto e : tokens)
36         cout << e << " ";
37      cout << endl;
38      return 0;
39  }
40
41  The split function is not shown here.
42
```

Program Output

```
one two three four
10 20 30 40 50
a b c d e f
```

10.8 Focus on Problem Solving and Program Design: A Case Study

As a programmer for the Home Software Company, you are asked to develop a function named `dollarFormat` that inserts commas and a $ sign at the appropriate locations in a `string` object containing an unformatted dollar amount. As an argument, the function should accept a reference to a `string` object. You may assume the `string` object contains a value such as 1084567.89. The function should modify the `string` object so it contains a formatted dollar amount, such as $1,084,567.89.

The code for the `dollarFormat` function is as follows:

```
void dollarFormat(string &currency)
{
   int dp;
   dp = currency.find('.');    // Find decimal point
   if (dp > 3)                 // Insert commas
```

```
    {
       for (int x = dp - 3; x > 0; x -= 3)
            currency.insert(x, ",");
    }
    currency.insert(0, "$");   // Insert dollar sign
}
```

The function defines an `int` variable named `dp`. This variable is used to hold the position of the unformatted number's decimal point. This is accomplished with the statement:

```
dp = currency.find('.');
```

The `string` class's `find` member function returns the position number in the string where the '.' character is found. An `if` statement determines if the number has more than three numbers preceding the decimal point:

```
if (dp > 3)
```

If the decimal point is at a position greater than 3, then the function inserts commas in the string with the following loop:

```
for (int x = dp - 3; x > 0; x -= 3)
    currency.insert(x, ",");
```

Finally, a $ symbol is inserted at position 0 (the first character in the `string`).

Program 10-24 demonstrates the function.

Program 10-24

```
1  // This program lets the user enter a number. The
2  // dollarFormat function formats the number as
3  // a dollar amount.
4  #include <iostream>
5  #include <string>
6  using namespace std;
7
8  // Function prototype
9  void dollarFormat(string &);
10
11 int main ()
12 {
13      string input;
14
15      // Get the dollar amount from the user.
16      cout << "Enter a dollar amount in the form nnnnn.nn : ";
17      cin >> input;
18      dollarFormat(input);
19      cout << "Here is the amount formatted:\n";
20      cout << input << endl;
21      return 0;
22 }
23
```

```
24  //************************************************************
25  // Definition of the dollarFormat function. This function    *
26  // accepts a string reference object, which is assumed       *
27  // to hold a number with a decimal point. The function       *
28  // formats the number as a dollar amount with commas and     *
29  // a $ symbol.                                               *
30  //************************************************************
31
32  void dollarFormat(string &currency)
33  {
34       int dp;
35
36       dp = currency.find('.');     // Find decimal point
37       if (dp > 3)                  // Insert commas
38       {
39           for (int x = dp - 3; x > 0; x -= 3)
40                currency.insert(x, ",");
41       }
42       currency.insert(0, "$");     // Insert dollar sign
43  }
```

Program Output with Example Input Shown in Bold

```
Enter a dollar amount in the form nnnnn.nn: 1084567.89 Enter
Here is the amount formatted:
$1,084,567.89
```

Review Questions and Exercises

Short Answer

1. What header file must you include in a program using character-testing functions such as `isalpha` and `isdigit`?
2. What header file must you include in a program using the character conversion functions `toupper` and `tolower`?
3. Assume `c` is a `char` variable. What value does `c` hold after each of the following statements executes?

Statement	Contents of `c`
`c = toupper('a');`	_____
`c = toupper('B');`	_____
`c = tolower('D');`	_____
`c = toupper('e');`	_____

4. Look at the following code. What value will be stored in `s` after the code executes?

```
char name[10];
int s;
strcpy(name, "Jimmy");
s = strlen(name);
```

5. What header file must you include in a program using string functions such as `strlen` and `strcpy`?

6. What header file must you include in a program using string/numeric conversion functions such as `atoi` and `atof`?
7. What header file must you include in a program using `string` class objects?
8. How do you compare `string` class objects?

Fill-in-the-Blank

9. The _____ function returns `true` if the character argument is uppercase.
10. The _____ function returns `true` if the character argument is a letter of the alphabet.
11. The _____ function returns `true` if the character argument is a digit.
12. The _____ function returns `true` if the character argument is a whitespace character.
13. The _____ function returns the uppercase equivalent of its character argument.
14. The _____ function returns the lowercase equivalent of its character argument.
15. The _____ file must be included in a program that uses character-testing functions.
16. The _____ function returns the length of a string.
17. To _____ two strings means to append one string to the other.
18. The _____ function concatenates two strings.
19. The _____ function copies one string to another.
20. The _____ function searches for a string inside of another one.
21. The _____ function compares two strings.
22. The _____ function copies, at most, *n* number of characters from one string to another.
23. The _____ function returns the value of a string converted to an integer.
24. The _____ function returns the value of a string converted to a long integer.
25. The _____ function returns the value of a string converted to a float.
26. The _____ function converts an integer to a string.

Algorithm Workbench

27. The following `if` statement determines whether choice is equal to `'Y'` or `'y'`:

    ```
    if (choice == 'Y' || choice == 'y')
    ```

    Simplify this statement by using either the `toupper` or `tolower` function.
28. Assume `input` is a `char` array holding a C-string. Write code that counts the number of elements in the array that contain an alphabetic character.
29. Look at the following array definition:

    ```
    char str[10];
    ```

    Assume `name` is also a `char` array, and it holds a C-string. Write code that copies the contents of `name` to `str` if the C-string in `name` is not too big to fit in `str`.

30. Look at the following statements:

```
char str[] = "237.89";
double value;
```

Write a statement that converts the string in `str` to a `double` and stores the result in `value`.

31. Write a function that accepts a pointer to a C-string as its argument. The function should count the number of times the character 'w' occurs in the argument and return that number.
32. Assume `str1` and `str2` are string class objects. Write code that displays "They are the same!" if the two objects contain the same string.

True or False

33. T F Character-testing functions, such as `isupper`, accept strings as arguments and test each character in the string.
34. T F If `toupper`'s argument is already uppercase, it is returned as is, with no changes.
35. T F If `tolower`'s argument is already lowercase, it will be inadvertently converted to uppercase.
36. T F The `strlen` function returns the size of the array containing a string.
37. T F If the starting address of a C-string is passed into a pointer parameter, it can be assumed that all the characters, from that address up to the byte that holds the null terminator, are part of the string.
38. T F C-string-handling functions accept as arguments pointers to strings (array names or pointer variables), or literal strings.
39. T F The `strcat` function checks to make sure the first string is large enough to hold both strings before performing the concatenation.
40. T F The `strcpy` function will overwrite the contents of its first string argument.
41. T F The `strcpy` function performs no bounds checking on the first argument.
42. T F There is no difference between "847" and 847.

Find the Errors

Each of the following programs or program segments has errors. Find as many as you can.

43.
```
char str[] = "Stop";
if (isupper(str) == "STOP")
   exit(0);
```

44.
```
char numeric[5];
int x = 123;
numeric = atoi(x);
```

45.
```
char string1[] = "Billy";
char string2[] = " Bob Jones";
strcat(string1, string2);
```

```
46. char x = 'a', y = 'a';
    if (strcmp(x, y) == 0)
       exit(0);
```

Programming Challenges

1. **String Length**

   Write a function that returns an integer and accepts a pointer to a C-string as an argument. The function should count the number of characters in the string and return that number. Demonstrate the function in a simple program that asks the user to input a string, passes it to the function, then displays the function's return value.



VideoNote
Solving the Backward String Problem

2. **Backward String**

   Write a function that accepts a pointer to a C-string as an argument and displays its contents backward. For instance, if the string argument is `"Gravity"` the function should display `"ytivarG"`. Demonstrate the function in a program that asks the user to input a string then passes it to the function.

3. **Word Counter**

   Write a function that accepts a pointer to a C-string as an argument and returns the number of words contained in the string. For instance, if the string argument is "Four score and seven years ago" the function should return the number 6. Demonstrate the function in a program that asks the user to input a string then passes it to the function. The number of words in the string should be displayed on the screen. *Optional Exercise:* Write an overloaded version of this function that accepts a `string` class object as its argument.

4. **Average Number of Letters**

   Modify the program you wrote for Programming Challenge 3 (Word Counter), so it also displays the average number of letters in each word.

5. **Sentence Capitalizer**

   Write a function that accepts a pointer to a C-string as an argument and capitalizes the first character of each sentence in the string. For instance, if the string argument is `"hello. my name is Joe. what is your name?"` the function should manipulate the string so that it contains `"Hello. My name is Joe. What is your name?"` Demonstrate the function in a program that asks the user to input a string then passes it to the function. The modified string should be displayed on the screen. *Optional Exercise:* Write an overloaded version of this function that accepts a `string` class object as its argument.

6. **Vowels and Consonants**

   Write a function that accepts a pointer to a C-string as its argument. The function should count the number of vowels appearing in the string and return that number.

   Write another function that accepts a pointer to a C-string as its argument. This function should count the number of consonants appearing in the string and return that number.

Demonstrate these two functions in a program that performs the following steps:

1. The user is asked to enter a string.
2. The program displays the following menu:
   A) Count the number of vowels in the string
   B) Count the number of consonants in the string
   C) Count both the vowels and consonants in the string
   D) Enter another string
   E) Exit the program
3. The program performs the operation selected by the user and repeats until the user selects E to exit the program.

7. Name Arranger

Write a program that asks for the user's first, middle, and last names. The names should be stored in three different character arrays. The program should then store, in a fourth array, the name arranged in the following manner: the last name followed by a comma and a space, followed by the first name and a space, followed by the middle name. For example, if the user entered `"Carol Lynn Smith"`, it should store `"Smith, Carol Lynn"` in the fourth array. Display the contents of the fourth array on the screen.

8. Sum of Digits in a String

Write a program that asks the user to enter a series of single-digit numbers with nothing separating them. Read the input as a C-string or a `string` object. The program should display the sum of all the single-digit numbers in the string. For example, if the user enters 2514, the program should display 12, which is the sum of 2, 5, 1, and 4. The program should also display the highest and lowest digits in the string.

9. Most Frequent Character

Write a function that accepts either a pointer to a C-string, or a `string` object, as its argument. The function should return the character that appears most frequently in the string. Demonstrate the function in a complete program.

10. `replaceSubstring` Function

Write a function named `replaceSubstring`. The function should accept three C-string or `string` object arguments. Let's call them *`string1`*, *`string2`*, and *`string3`*. It should search *`string1`* for all occurrences of *`string2`*. When it finds an occurrence of *`string2`*, it should replace it with *`string3`*. For example, suppose the three arguments have the following values:

`string1`: "the dog jumped over the fence"
`string2`: "the"
`string3`: "that"

With these three arguments, the function would return a `string` object with the value "that dog jumped over that fence." Demonstrate the function in a complete program.

11. Case Manipulator

Write a program with three functions: `upper`, `lower`, and `reverse`. The `upper` function should accept a pointer to a C-string as an argument. It should step through each character in the string, converting it to uppercase. The `lower` function, too, should accept a pointer to a C-string as an argument. It should step through each character in the string,

converting it to lowercase. Like `upper` and `lower`, `reverse` should also accept a pointer to a string. As it steps through the string, it should test each character to determine whether it is uppercase or lowercase. If a character is uppercase, it should be converted to lowercase. Likewise, if a character is lowercase, it should be converted to uppercase.

Test the functions by asking for a string in function `main`, then passing it to them in the following order: `reverse`, `lower`, and `upper`.

12. Password Verifier

Imagine you are developing a software package that requires users to enter their own passwords. Your software requires that users' passwords meet the following criteria:

- The password should be at least six characters long.
- The password should contain at least one uppercase and at least one lowercase letter.
- The password should have at least one digit.

Write a program that asks for a password then verifies that it meets the stated criteria. If it doesn't, the program should display a message telling the user why.

13. Date Printer

Write a program that reads a string from the user containing a date in the form mm/dd/yyyy. It should print the date in the form March 12, 2018.

14. Word Separator

Write a program that accepts as input a sentence in which all of the words are run together, but the first character of each word is uppercase. Convert the sentence to a string in which the words are separated by spaces and only the first word starts with an uppercase letter. For example, the string "StopAndSmellTheRoses." would be converted to "Stop and smell the roses."

15. Character Analysis

For this programming problem, download the file text.txt. Write a program that reads the file's contents and determines the following:

- The number of uppercase letters in the file
- The number of lowercase letters in the file
- The number of digits in the file

16. Pig Latin

Write a program that reads a sentence as input and converts each word to "Pig Latin." In one version, to convert a word to Pig Latin, you remove the first letter and place that letter at the end of the word. Then you append the string "ay" to the word. Here is an example:

English: I SLEPT MOST OF THE NIGHT
Pig Latin: IAY LEPTSAY OSTMAY FOAY HETAY IGHTNAY

17. Morse Code Converter

Morse code is a code where each letter of the English alphabet, each digit, and various punctuation characters are represented by a series of dots and dashes. Table 10-10 shows part of the code.

Write a program that asks the user to enter a string then converts that string to Morse code.

Table 10-10 Morse Code

Character	Code	Character	Code	Character	Code	Character	Code
space	*space*	6	-....	G	--.	Q	--.-
comma	--.--	7	--...	H	....	R	.-.
period	.-.-.-	8	---..	I	..	S	...
question mark	..--..	9	----.	J	.---	T	-
0	-----	A	.-	K	-.-	U	..-
1	.----	B	-...	L	.-..	V	...-
2	..---	C	-.-.	M	--	W	.--
3	...--	D	-..	N	-.	X	-..-
4	....-	E	.	O	---	Y	-.--
5	.....	F	..-.	P	.--.	Z	--..

18. Phone Number List

Write a program that has an array of at least 10 `string` objects that hold people's names and phone numbers. You may make up your own strings, or use the following:

```
"Alejandra Cruz, 555-1223"
"Joe Looney, 555-0097"
"Geri Palmer, 555-8787"
"Li Chen, 555-1212"
"Holly Gaddis, 555-8878"
"Sam Wiggins, 555-0998"
"Bob Kain, 555-8712"
"Tim Haynes, 555-7676"
"Warren Gaddis, 555-9037"
"Jean James, 555-4939"
"Ron Palmer, 555-2783"
```

The program should ask the user to enter a name or partial name to search for in the array. Any entries in the array that match the string entered should be displayed. For example, if the user enters `"Palmer"` the program should display the following names from the list:

```
Geri Palmer, 555-8787
Ron Palmer, 555-2783
```

19. Check Writer

Write a program that displays a simulated paycheck. The program should ask the user to enter the date, the payee's name, and the amount of the check (up to $10,000). It should then display a simulated check with the dollar amount spelled out, as shown here:

```
                                                Date: 11/24/2021


Pay to the Order of: John Phillips                    $1920.85

One thousand nine hundred twenty and 85 cents
```

Be sure to format the numeric value of the check in fixed-point notation with two decimal places of precision. Be sure the decimal place always displays, even when the number is zero or has no fractional part. Use either C-strings or `string` class objects in this program.

Input Validation: Do not accept negative dollar amounts, or amounts over $10,000.

20. Lottery Statistics

To play the PowerBall lottery, you buy a ticket that has five numbers in the range of 1–69, and a "PowerBall" number in the range of 1-26. (You can pick the numbers yourself, or you can let the ticket machine randomly pick them for you.) Then, on a specified date, a winning set of numbers are randomly selected by a machine. If your first five numbers match the first five winning numbers in any order, and your PowerBall number matches the winning PowerBall number, then you win the jackpot, which is a very large amount of money. If your numbers match only some of the winning numbers, you win a lesser amount, depending on how many of the winning numbers you have matched.

In the student sample programs for this book, you will find a file named pbnumbers.txt, containing the winning lottery numbers that were selected between February 3, 2010 and May 11, 2016 (the file contains 654 sets of winning numbers). Here is an example of the first few lines of the file's contents:

```
17 22 36 37 52 24
14 22 52 54 59 04
05 08 29 37 38 34
10 14 30 40 51 01
07 08 19 26 36 15
and so on ...
```

Each line in the file contains the set of six numbers that were selected on a given date. The numbers are separated by a space, and the last number in each line is the PowerBall number for that day. For example, the first line in the file shows the numbers for February 3, 2010, which are 17, 22, 36, 37, 52, and the PowerBall number 24.

Write one or more programs that work with this file to perform the following:

- Display the 10 most common numbers, ordered by frequency.
- Display the 10 least common numbers, ordered by frequency.
- Display the 10 most overdue numbers (numbers that haven't been drawn in a long time), ordered from most overdue to least overdue.
- Display the frequency of each number 1-69, and the frequency of each Powerball number 1-26.